

THE OPEN SOURCE AUDIT

A Capstone Project for OSS NGMC Course

LINUX KERNEL

Student Name	Anjali Sharma
Reg. Number	24BAI10310
Course	Open source Software - OSS NGMC
Unit Coverage	Units 1 - 5
Max Marks	100
Submission	VITyarthi Portal

TABLE OF CONTENTS

A1. The Problem the Linux Kernel Was Created to Solve	3
A2. The License — What It Actually Says	5
A3. The Ethics of Open Source	7



1. Introduction

This report is a structured audit of the Linux Kernel — the most consequential piece of open-source software ever created. Rather than summarising what the kernel does technically, this audit examines why it exists, who built it, what freedoms its license guarantees, and how it has shaped the philosophy and practice of software development worldwide.

The Linux Kernel is not just a program. It is a philosophical statement — a proof that collaborative, transparent, community-driven development can outperform any proprietary effort. From smartphones to supercomputers, from web servers to spacecraft, the Linux Kernel is the silent foundation beneath nearly all modern computing.

Alongside this written analysis, five shell scripts are documented in Section 6. These scripts demonstrate practical Ubuntu/Debian Linux skills — from system inspection to log analysis — and connect hands-on command-line knowledge to the open-source philosophy explored in the report.

2. Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem the Linux Kernel Was Created to Solve

To understand why the Linux Kernel matters, you have to go back to 1991 and understand what the computing world looked like — and who was being left out of it.

The World Before Linux

In the late 1980s and early 1990s, Unix was the gold standard of operating systems. It was powerful, stable, and designed for serious computing. But it was locked behind commercial licences that cost thousands of dollars. A student or hobbyist who wanted to learn real systems programming had essentially no affordable path. Microsoft's MS-DOS existed, but it was primitive, single-tasking, and architecturally uninteresting.

MINIX — a small Unix-like teaching OS created by Professor Andrew Tanenbaum — existed, but Tanenbaum had deliberately kept it simple and restrictive. Its licence did not allow redistribution of modifications, and its microkernel architecture made it unsuitable for serious use. Students could read it but could not meaningfully build on it.

The GNU Project, started by Richard Stallman in 1983, had been building free alternatives to Unix tools for nearly a decade. By 1991 it had a compiler (GCC), a shell (Bash), text editors, and dozens of other utilities. But it was missing one critical component: a kernel. The GNU Hurd kernel was in development but years away from being usable. There was a complete set of free tools with no free engine to run them on.

Linus Torvalds — A Student With a Problem

Linus Benedict Torvalds was a 21-year-old computer science student at the University of Helsinki in Finland. He had bought an IBM 80386 PC — a powerful machine for its time — and he wanted to explore its full capabilities. The only OS he could run was MS-DOS, which was far below the machine's potential. He could not afford a commercial Unix licence.

He began by reading the MINIX source code and using it as a reference, but he quickly grew frustrated with its limitations. He wanted terminal emulation so he could log into the university's Unix servers, he wanted to use disk drives properly, and he wanted multitasking. MINIX could not give him these things in the way he envisioned.

So he started writing his own. Not with grand ambitions — in his now-famous message to the comp.os.minix newsgroup on 25 August 1991, Torvalds wrote: 'I'm doing a (free) operating system (just a hobby, won't be big and professional like gnu).' He called it Linux.

The Decision to Share — Why Open Source?

Torvalds did not have to share his work. He could have kept it private or sold it. But he made a different choice: he posted the source code to the internet and invited others to improve it. This decision was informed by three things.

- **Practical necessity:** He needed hardware drivers for devices he did not own. Opening the source meant people with those devices could write and contribute the drivers themselves.
- **The hacker culture:** The early internet was built on a culture of sharing code and knowledge freely. Torvalds had grown up in this culture and believed in it.
- **The GNU ecosystem:** His kernel needed GCC to compile and GNU tools to run. Releasing under a compatible free licence was the natural and respectful thing to do within that ecosystem.

In 1992, Torvalds relicensed Linux under the GNU General Public License version 2 (GPL v2). This was a defining moment. The GPL meant that anyone who distributed Linux — or any software built on it — had to share their source code under the same terms. The kernel could never be locked away.

The Consequence — A Problem Bigger Than One Student

What started as one student solving his own problem became the solution to a problem shared by the entire world. Within months, developers across the globe were contributing patches. By the mid-1990s, Linux was powering servers. By the 2000s, it was inside Android phones. Today, the Linux Kernel runs on over 96% of the world's top one million web servers, all 500 of the world's fastest supercomputers, and billions of Android devices. The student who just wanted a free Unix has given the world its most important piece of software.

A2. The License — What It Actually Says

The GNU General Public License Version 2 (GPL v2)

The Linux Kernel is distributed under the GNU General Public License version 2. Reading the actual licence text — available at [kernel.org](https://www.kernel.org) — reveals not just legal terms but a philosophy of software freedom expressed in binding legal language.

The Four Freedoms of Free Software

Richard Stallman defined four essential freedoms that any truly free software must provide. The GPL v2 was designed to protect all four:

- **Freedom 0:** The freedom to run the program for any purpose.
- **Freedom 1:** The freedom to study how the program works and adapt it. Access to the source code is a precondition.
- **Freedom 2:** The freedom to redistribute copies to help others.
- **Freedom 3:** The freedom to distribute copies of your modified versions. This lets the whole community benefit from your improvements.

The Linux Kernel grants all four freedoms. You can download it, read every line of code, modify it to fit your needs, and redistribute those modifications — provided you do so under the same GPL v2 terms. This last condition is the heart of what makes the GPL different from more permissive licences.

Can a Company Modify and Sell Linux Without Sharing?

This is one of the most important questions the GPL answers — and it answers it clearly: No.

Section 2 of the GPL v2 states that if you distribute a modified version of a GPL-licensed programme, you must release the complete corresponding source code under the same GPL terms. This is the 'copyleft' mechanism. It works like a legal mirror: the freedom you received must be passed on to everyone who receives your work.

A company can absolutely build commercial products on Linux. Red Hat has done this profitably for decades. Android is built on the Linux Kernel. But those companies cannot take the kernel, modify it, ship it in a product, and keep their changes private. The source must be shared. Red Hat makes money from services and support, not from keeping code secret — and the GPL forces that business model on anyone who builds with Linux.

GPL vs MIT — A Critical Comparison

Dimension	GNU GPL v2	MIT Licence
Copyleft	Strong copyleft — derivatives must also be GPL	None — derivatives can be proprietary
Commercial use	Allowed, but source must be shared	Fully allowed, no restrictions
Patent rights	Implicit patent grant	No explicit patent clause

Viral nature	Yes — if you distribute, you must open source	No — just keep the copyright notice
Best for	Projects where community freedom is the priority	Libraries and tools where maximum adoption matters

If I were building a software project today, I would choose the licence based on intention. For a library that I want developers everywhere to adopt — like a utility function set — MIT is pragmatic because it removes all barriers to adoption. But for a project where I believe the community should always benefit from improvements — as Torvalds believed with Linux — I would choose GPL v2. The copyleft clause ensures the commons grows, rather than being enclosed by commercial interests.

Free as in Beer vs Free as in Freedom

This distinction, first articulated by Richard Stallman, is fundamental to understanding the open-source world. 'Free as in free beer' means you pay no money — a freeware programme might be free to download but its source code is closed and you cannot modify it. 'Free as in freedom' means you have the liberty to do what you want with the software.

The Linux Kernel is both, but the second kind of freedom is what matters. Consider this concrete example: Android, which is built on the Linux Kernel, is free to download on your phone. That is free as in beer. But because the kernel is GPL v2, Google was legally required to publish their kernel modifications for Android — and they have, at source.android.com. That is free as in freedom: anyone can take Google's Android kernel code, study it, improve it, or use it in their own device.

A3. The Ethics of Open Source

Should All Software Be Open Source?

This question has no clean answer, and the fact that it does not is itself instructive about the complexity of software ethics.

The case for universal open source: Knowledge has historically been most useful to humanity when it is freely shared. Mathematics, science, and literature have all advanced through open exchange. Software is a form of knowledge — it encodes solutions to problems. If a hospital system, a voting machine, or an AI model affects millions of people, those people arguably have a right to know how it works. Security in closed systems is 'security through obscurity' — a principle widely condemned by security experts, because it means only the attacker who finds a vulnerability knows about it. In open systems, thousands of eyes can review the code, and flaws are found and fixed faster.

The case against mandating universal open source: Commercial software development requires significant investment — not just in code, but in design, testing, customer support, and infrastructure. Proprietary licences fund that investment. A small team building a specialised medical device software cannot necessarily sustain themselves on open-source business models. Innovation in some domains — pharmaceuticals, aerospace — requires protecting intellectual property long enough to recoup research costs. Furthermore, open source does not automatically mean better or more secure — the Heartbleed bug lived in OpenSSL, a widely reviewed open-source library, for over two years.

My view is that the answer is contextual. Infrastructure software — operating systems, compilers, networking tools — should be open source because it forms a commons that everyone depends on. Application software and specialised commercial tools can reasonably be proprietary. The important thing is that the choice is made with awareness of its implications, not by default.

Is It Ethical to Profit from Community Labour?

This is perhaps the most uncomfortable question in the open-source ecosystem, and it is one that the Linux community has wrestled with directly.

Companies like Red Hat, IBM, Google, and Meta have each contributed significant resources to Linux. Red Hat employs some of the most prolific kernel contributors. Google maintains the Android kernel and has contributed enormous resources to LLVM, Kubernetes, and TensorFlow. These companies also profit from open-source software — sometimes enormously. Is that ethical?

I think the answer depends on what 'exploitation' means in context. If a company takes from the commons and gives nothing back — using GPL code in products without contributing improvements, or lobbying against the open-source ecosystem — that is ethically problematic. But if a company uses open-source tools as part of a product, contributes upstream, pays engineers to work on open-source projects, and operates within the spirit of the licence, that is not exploitation — it is the intended model. The GPL was designed to allow commercial use

while ensuring reciprocity. The ethics break down when reciprocity is absent.

The more troubling case is smaller projects — the maintainer of an obscure but critical library who is relied upon by millions of applications, but who has no corporate backing and maintains it in their spare time. This is a genuine structural problem in open source: the extraction is real, even if it is not malicious.

Standing on the Shoulders of Giants

Isaac Newton wrote: 'If I have seen further, it is by standing on the shoulders of giants.' In no field is this more literally true than in software, and in no corner of software more than open source.

Every application on your phone rests on Android, which rests on the Linux Kernel, which was compiled by GCC, which runs on hardware drivers written in C, which is a language whose concepts trace back to research done at AT&T; Bell Labs in the 1970s — code and ideas shared in academic papers and technical reports that preceded the open-source movement by decades.

Open source does not undermine original innovation — it accelerates it. By not reinventing the foundations, each new developer can start from a higher baseline. The Linux Kernel itself would not have been possible without Tanenbaum's MINIX to learn from, Stallman's GNU tools to build with, and the accumulated knowledge of Unix shared in academic literature. Innovation in software is almost always recombination — seeing a new way to assemble existing ideas. Open source makes that recombination possible at civilisational scale.

3. Part B — Linux Footprint (Unit 2)

Installation on Ubuntu/Debian

On Ubuntu and Debian-based systems, the Linux Kernel is installed and managed through the APT package management system. Most users never install the kernel manually — it arrives as part of the distribution install and is updated automatically. But understanding how it is installed reveals the entire Linux package ecosystem.

Package names and installation commands:

```
sudo apt update
sudo apt install linux-generic
sudo apt install linux-headers-$(uname -r)
```

The *linux-generic* metapackage pulls in the latest stable kernel image and headers for the running architecture. Individual kernel version packages are named like *linux-image-6.5.0-44-generic*. The kernel image itself, boot loader, and initial RAM disk are installed during this process.

Key Directories on the Filesystem

Directory	Contents	Significance
/boot	vmlinuz, initrd.img, System.map, config-*	The compressed kernel image and boot loader files
/lib/modules/\$(uname -r)	Loadable kernel modules (.ko files)	Drivers and features loaded dynamically at runtime
/etc/modprobe.d/	Module configuration files	Controls module load options and blacklisting
/proc/	Virtual filesystem — kernel runtime state	Exposes live kernel data: /proc/version, /proc/cpuinfo
/sys/	sysfs — hardware and driver information	Allows userspace to query and configure kernel subsystems
/usr/src/linux-headers-*	Kernel header files for compilation	Required for building external kernel modules
/var/log/kern.log	Kernel message log	Stores kernel ring buffer messages persistently

User, Group, and Security Context

The Linux Kernel itself does not run as a user in the conventional sense — it is the privileged layer below all user processes. However, processes it spawns and manages do run under specific users:

- **root (UID 0):** The kernel runs with full privilege. Kernel threads visible in `ps` (like `kthreadd`, `ksoftirqd`) show `root` as their owner.
- **nobody / www-data:** The kernel's security model enforces that processes should run with minimal privilege. A web server runs as `www-data`, not `root`, so a compromised server cannot access the whole system.
- **Kernel modules:** Kernel modules run in ring 0 — the highest privilege level. This is why only `root` can load modules (`insmod/modprobe`), and why code quality matters enormously in drivers.

The security implications of this are profound. The open-source nature of the kernel means security researchers worldwide can audit every privilege boundary. When the Dirty COW vulnerability (CVE-2016-5195) was found — a race condition in the kernel's memory subsystem that allowed privilege escalation — it was patched within days of public disclosure, and the fix was pushed to millions of systems via `apt upgrade`.

Service Management and Commands

The kernel itself is not managed as a service in the `systemd` sense — it is loaded by the bootloader (GRUB2) before any service manager starts. However, kernel-related management happens through several interfaces:

```
# Check currently running kernel version
uname -r

# List all installed kernel versions
dpkg -l | grep linux-image

# Load a kernel module
sudo modprobe module_name

# Check loaded modules
lsmod

# View kernel messages live
dmesg | tail -50

# View kernel log file
sudo tail -f /var/log/kern.log
```

Update Model — How Security Patches Reach Your Machine

The Linux Kernel's update model on Ubuntu is one of the most impressive security pipelines in software. When a vulnerability is found:

- The fix is developed by kernel maintainers and merged into the stable kernel tree at `kernel.org`.
- Canonical's kernel team backports the fix to the Ubuntu kernel version and packages it as an APT update.
- The update is pushed to Ubuntu's security repositories and flagged as a security update.

- apt upgrade (or unattended-upgrades for automatic updates) downloads and installs the new kernel package.
- A system reboot loads the new kernel. The old kernel remains available as a fallback in GRUB2.

This pipeline — from upstream kernel.org to your laptop — typically takes 24–72 hours for critical vulnerabilities. No proprietary OS can match this transparency: every patch is publicly visible in the git commit history, every CVE is linked to the exact code change that fixes it.

4. Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies — What Linux Builds Upon

The Linux Kernel has remarkably few external dependencies by design — it is a self-contained piece of software that must boot on bare metal. But building and using the kernel requires a rich ecosystem of tools:

- **GCC (GNU Compiler Collection):** The primary compiler for building the kernel. The kernel is written in C and (since 2022) limited Rust. GCC translates this code into machine instructions.
- **GNU Make:** The build system that orchestrates kernel compilation across hundreds of source directories.
- **GNU Binutils:** Linkers, assemblers, and other binary tools needed during kernel compilation.
- **OpenSSL / libcrypto:** Kernel modules that implement cryptographic operations link against these.
- **LLVM/Clang:** An alternative compiler toolchain increasingly used to build the kernel, especially for security analysis and Rust support.

What the Linux Kernel Has Enabled

The Linux Kernel has not just run the world — it has inspired an entire generation of software systems and directly enabled many of the most important technologies in use today:

- **Android:** Google's mobile OS is built directly on a modified Linux Kernel. The Android Open Source Project (AOSP) maintains a fork with mobile-specific additions.
- **Container technology:** Docker, Kubernetes, and all modern container orchestration rely on Linux Kernel features — specifically namespaces and cgroups — to isolate processes without full virtualisation.
- **Chrome OS:** Google's Chromebook OS runs a Linux Kernel with a custom init system and Chromium browser.
- **Embedded Linux:** Routers, smart TVs, IoT devices, and industrial controllers run stripped-down Linux kernels. The Raspberry Pi runs Linux.
- **eBPF:** An in-kernel virtual machine that has enabled a revolution in observability and networking. Tools like Cilium and Falco — used in production at Netflix and Cloudflare — are entirely eBPF-based.

The Linux Kernel in the LAMP Stack

LAMP — Linux, Apache, MySQL, PHP — is the classic open-source web stack. Linux is not just one component of this stack: it is the environment in which all other components run. Every Apache process, every MySQL query, every PHP execution happens through system calls to the Linux Kernel.

The kernel manages memory allocation for Apache's worker processes, schedules CPU time between database queries, handles TCP/IP networking for all HTTP traffic, and enforces the file permissions that protect configuration files and user data. Without the kernel, LAMP does not exist. This is why understanding the kernel is foundational to understanding web infrastructure.

Community and Governance

The Linux Kernel project is arguably the largest collaborative engineering project in human history. Its governance model is unusual and worth understanding in detail.

- **Linus Torvalds (BDFL):** Torvalds remains the final decision-maker on what enters the mainline kernel. He has delegated responsibility for subsystems to trusted lieutenants but retains ultimate authority.
- **Subsystem maintainers:** Each major area of the kernel (networking, filesystems, security, graphics) has a designated maintainer who reviews patches for their area.
- **The merge window:** Twice per year, Torvalds opens a two-week merge window during which major new features are accepted. Bug-fix releases (stable series) follow on a regular schedule.
- **LKML (Linux Kernel Mailing List):** The primary communication channel. All patches are submitted and discussed via email — a deliberately low-barrier, text-based system.
- **Linux Foundation:** A nonprofit that provides infrastructure, legal support, and coordinates corporate contributions. Member companies include Google, Meta, Intel, Samsung, and Microsoft.
- **Kernel Summit:** An annual in-person gathering of senior kernel developers where architectural decisions and community issues are discussed.

A 2023 Linux Foundation report found that over 4,300 developers from more than 600 companies contributed to the Linux Kernel in a recent development cycle. Roughly one third of contributions come from individual volunteers; the rest from paid corporate engineers. This balance — between community idealism and corporate practicality — is what makes Linux sustainable.

5. Part D — Open Source vs Proprietary (Critical Analysis)

Comparison: Linux Kernel vs Windows NT Kernel

Dimension	Linux Kernel (Open Source)	Windows NT Kernel (Proprietary)
Cost	Free — zero licence cost for any use	Bundled with Windows licence; significant cost for servers and enterprise
Security (code audit)	Full source available; thousands of global researchers audit it daily	Source closed; Microsoft's internal team and select partners audit it
Support and reliability	Community + paid support from Red Hat, Canonical, SUSE. LTS kernels maintained for 6+ years	Guaranteed Microsoft support; Enterprise support contracts available. Clear end-of-life dates
Freedom to modify	Complete — you can patch, fork, and redistribute under GPL v2	None — you cannot legally modify or redistribute Windows
Community vs corporate control	Distributed governance; Torvalds + subsystem maintainers + Linux Foundation	Single corporate control by Microsoft; roadmap driven by business strategy
Driver support	Excellent for server and developer hardware; historically weaker for consumer peripherals	Excellent across all consumer hardware; Microsoft WHQL certification process
Performance	World-class on servers; powers all top 500 supercomputers	Excellent on desktop; server performance competitive but not dominant
Transparency	Every commit, every bug, every discussion is public	Internal processes hidden; security bulletins without full technical disclosure

Verdict — Real-World Deployment Recommendation

For any server, cloud, or infrastructure deployment, I would recommend Linux without hesitation. The combination of zero licence cost, complete transparency, world-class performance, and the ability to customise the kernel to specific workloads makes it objectively superior for these use cases. The evidence is not theoretical — AWS, Google Cloud, and Azure all run primarily on Linux. The internet runs on Linux. The world's science runs on Linux.

For desktop use, the answer is more nuanced. Windows has genuine advantages in consumer hardware driver support, gaming, and Microsoft Office compatibility that Linux has not fully closed. But for developers, researchers, and technically inclined users, Linux on the desktop is genuinely excellent — and getting better with every release.

Would I Contribute?

Yes — and studying this audit has made me understand both what that means and why it matters. Contributing to the Linux Kernel is famously demanding. Linus Torvalds is known for blunt and sometimes harsh code reviews. The standards for correctness, documentation, and consistency are extremely high.

But those standards exist for good reason. Code that enters the kernel runs on billions of devices. A bug in a filesystem driver can corrupt data worldwide. The demanding review process is not gatekeeping for its own sake — it is the quality assurance mechanism for the most consequential software in the world. I would want to contribute to that. Starting small — a documentation fix, a driver improvement for hardware I own — and working up to something meaningful. That path is open to anyone. That is the point.

6. Shell Script Documentation (Unit 2 & 5)

Five shell scripts were written for this project, each targeting Ubuntu/Debian Linux and demonstrating specific shell scripting concepts as required by the course rubric. All scripts are submitted as separate .sh files in the GitHub repository.

Script 1: System Identity Report

File: script1_system_identity.sh

Description: This script acts as a welcome screen for the Linux system, displaying the distro name (read from /etc/os-release), kernel version via `uname -r`, current logged-in user via `whoami`, home directory from `$HOME`, system uptime via `uptime -p`, current date and time, and a message about the kernel's GPL v2 licence.

Shell concepts used: Variables, echo, command substitution (`$()`), string formatting, if-file check, cut, tr.

Key code excerpt:

```
KERNEL=$(uname -r)
USER_NAME=$(whoami)
UPTIME=$(uptime -p)
DISTRO=$(grep PRETTY_NAME /etc/os-release | cut -d= -f2 | tr -d ' ')
echo "Kernel : $KERNEL"
echo "Distro : $DISTRO"
```

Script 2: FOSS Package Inspector

File: script2_package_inspector.sh

Description: This script checks whether Linux kernel-related packages (linux-generic, linux-headers-generic, gcc, make, git) are installed using `dpkg -l`. For each installed package, it retrieves the version and description. A case statement then prints an open-source philosophy note for each package.

Shell concepts used: If-then-else, for loop over array, case statement, `dpkg -l`, `apt-cache`, `awk`, `grep`, pipe.

Key code excerpt:

```
if dpkg -l "$PKG" &>/dev/null; then
echo "[INSTALLED] $PKG"
dpkg -l "$PKG" | grep '^ii' | awk '{print $3}'
fi
case $PKG in
gcc) echo 'The compiler that builds the free kernel' ;;
git) echo 'Built by Linus when proprietary tools failed' ;;
esac
```

Script 3: Disk and Permission Auditor

File: script3_disk_permission_auditor.sh

Description: This script loops through key system directories (/etc, /var/log, /home, /usr/bin, /tmp, /boot, /lib/modules) and for each reports: permissions and ownership using ls -ld + awk, and disk usage using du -sh + cut. A second section audits Linux-kernel-specific directories, and a third reads /proc/sys/kernel parameters.

Shell concepts used: For loop over array, if-directory check, ls -ld, awk, du -sh, cut, /proc filesystem access.

Key code excerpt:

```
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")
for DIR in "${DIRS[@]}; do
if [ -d "$DIR" ]; then
PERMS=$(ls -ld "$DIR" | awk '{print $1,$3,$4}')
SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
echo "$DIR => $PERMS | $SIZE"
fi
done
```

Script 4: Log File Analyzer

File: script4_log_analyzer.sh

Description: This script accepts a log file path (\$1) and search keyword (\$2, defaulting to 'error') as arguments. It uses a while-read loop to read each line, and an if-then inside it to count keyword occurrences (case-insensitive grep). A do-while style retry loop handles empty files. The last 5 matching lines are printed using grep + tail. Particularly useful with /var/log/kern.log for kernel message analysis.

Shell concepts used: While-read loop, if-then with counter, command-line arguments (\$1, \$2), grep -iq, tail, do-while style retry with sleep.

Key code excerpt:

```
while IFS= read -r LINE; do
if echo "$LINE" | grep -iq "$KEYWORD"; then
COUNT=$((COUNT + 1))
fi
done < "$LOGFILE"
grep -i "$KEYWORD" "$LOGFILE" | tail -5
```

Script 5: Open Source Manifesto Generator

File: script5_manifesto_generator.sh

Description: This is the most creative script. It prompts the user with three questions — an OSS tool they use daily, what freedom means to them, and something they would build and share freely. It then composes a personalised open-source philosophy paragraph using string concatenation and saves it to manifesto_\$(whoami).txt using > and >>. The alias concept is demonstrated in comments. The file is displayed with cat.

Shell concepts used: read for user input, string concatenation, > and >> file writing, date command, cat, aliases concept in comments, \$(whoami) for dynamic filename.

Key code excerpt:

```
read -p "Name an OSS tool you use: " TOOL
read -p "What does freedom mean? " FREEDOM
OUTPUT="manifesto_$(whoami).txt"
echo "Every day I use $TOOL..." > "$OUTPUT"
echo "To me this means $FREEDOM." >> "$OUTPUT"
cat "$OUTPUT"
```

7. Conclusion

Auditing the Linux Kernel has been an exercise in understanding how one person's practical frustration — a student who could not afford a Unix licence — became the foundation of the modern computing world. The Linux Kernel is not merely a technical achievement, impressive as that is. It is proof of a philosophical proposition: that collaborative, transparent, community-driven development can produce software of higher quality, greater security, and more enduring value than any proprietary alternative.

The GPL v2 licence is the legal embodiment of that philosophy. It does not just permit freedom — it enforces it. By requiring that all derivative work remain open, it ensures the commons grows rather than being enclosed. Every device that runs Android, every website served from an Apache/Nginx server, every data query processed by MySQL on Linux — all of it rests on a foundation that Linus Torvalds chose to give away.

The ethical questions raised by open source — about corporate exploitation, about the sustainability of volunteer labour, about whether all software should be free — do not have clean answers. But engaging with them seriously is part of what it means to be a software professional. The tools we build sit on top of work done by people who made choices about openness and sharing. We owe it to them, and to those who will build on our work, to think carefully about the choices we make.

Every line of code written in the coming decades will run on Linux, or on a system shaped by Linux. Understanding the kernel — not just how to use it, but why it exists, what values it represents, and how it is sustained — is not optional knowledge for a software professional. It is the foundation.

Submitted by Anjali Sharma | Reg. No: 24BAI10310 | Course: Open Source Software| VITyarthi OSS
NGMC Capstone